Training for Constraining: speaker notes

Levi Katyal Sledd and Vijay Singh · Devcon 8 workshop · notes for `slides/workshop.pdf` as of 28 October 2026

How to read. The notes run continuously in slide order. Each entry shows the slide in its final click state beside the talking track (**Say**), followed by stage directions (**Do**) and backup answers or cautions (**If asked**). *Clicks* counts the advances inside a slide; *start* is minutes into a 50-minute run. Section-title slides appear as headings.

Core message. A proof system can be perfectly sound while the application is broken, because the proof is about the constraints that were written, not the relation that was meant.

Timing plan (a proposal; calibrate in rehearsal)

Section	Slides	Start	Length	Who
Opening and motivation	1–4	0:00	3:30	
Circuits and constraints	5–12	3:30	8:45	
Patterns, Example 0, Example 1	13–21	12:15	7:45	
Exercise 2 (selector, pairs)	22–24	20:00	5:00	
Example 3 (range, foreign field)	25–30	25:00	6:15	
Deep dive: gnark fake GLV	31–37	31:15	7:00	
Finding underconstraints, demo	38–43	38:15	6:00	
Verification infrastructure, AI	44–47	44:15	3:15	
Wrap-up and questions	48–49	47:30	2:30	

The two `\note` timings inside `workshop.tex` (slides 12 and 14) predate Levi's revision and are superseded by this plan. **If behind schedule,** compress slides 9, 27 and 29 to one sentence each, and protect the exercise (23–24), the fake-GLV arc (32–37), the demo (42) and the close (49).

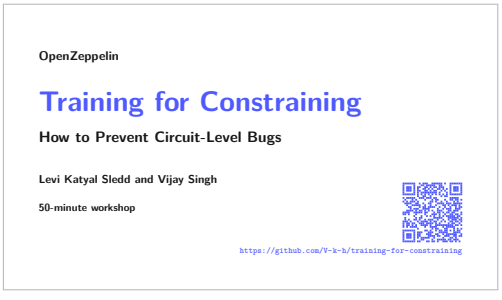
Before presenting

1. **Make the repository public.** The QR codes on slides 1 and 49 point to the repository, which returns 404 to the audience while it is private.
2. **Confirm the slot length.** The title slide says 50 minutes; Devcon 8 lists workshops at 60. With 60, give the extra time to the exercise and to questions.
3. **Remove drafting text printed on slides.** Slide 35 ("supplied HTML case study"), slide 37 ("verify precise affected paths and releases"), slide 51 ("Supplied reference: fake-glv-underconstraint.html" and "Check the release advisory"). Replacement facts, checked against gnark's source, are in the notes for slides 35 and 37.
4. **Slide 4, Airbender row.** A comment in the source questions whether these findings are circuit underconstraints. Verify against the report or drop the row.
5. **Preflight the demo on the presenting laptop**, offline: `make all` and `make lean` from the repository root, with a large terminal font.
6. **Slide 7** has an empty final click; click through it.
7. **Commit** `notes/smt-ring-theory.pdf` if you mention it on slide 43.

Closing line (slide 49): *A prover can replace every private witness and every hint. Only enforced constraints bind the public claim. Check the assumptions of your dependencies and compilers, not only your own code.*

1 Title

start **0:00** clicks **0** pdf page 1



Welcome. One sentence each to introduce yourselves; this is a hands-on session, so keep it short.

The claim for the next fifty minutes: a SNARK can be perfectly sound and the application still broken, because the proof is about the constraints you wrote, not the relation you meant.

Plan: four small examples you can check by hand, one real bug in a widely used library, and the tools that find these bugs.

Do. Point at the QR code: everything we run today is in this repository and needs only Python 3.10. Pause a few seconds

for phones.

2 Outline

start **0:40** clicks **0** pdf page 2



One thread runs through every section: write the intended relation, build an adversarial witness, watch the constraints accept it, repair, then ask how you would have found it.

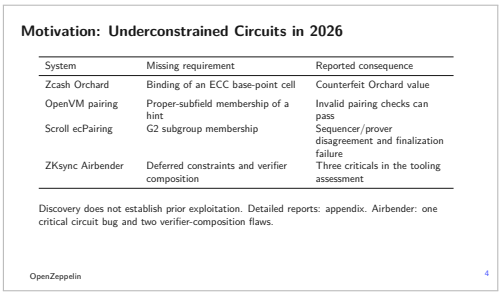
Two interactive moments: a two-minute pair exercise in the middle and a live demo near the end.

Do. Do not read the list aloud.

Slide 3, section title. *Motivation.* One breath, then advance.

4 Motivation: Underconstrained Circuits in 2026

start **1:00** clicks **0** pdf page 4



Read the middle column, not the first. Every row is a property somebody assumed and nobody enforced.

Pick one row and tell it properly. Orchard: an elliptic-curve base-point cell was not bound, and the reported consequence was counterfeit Orchard value. Then name the shape of the others: OpenVM did not check that a hint lay in the right subfield; Scroll did not check G2 subgroup membership.

Mature, audited systems, all this year, the same shape. Then the caveat on the slide: discovery does not establish prior exploitation.

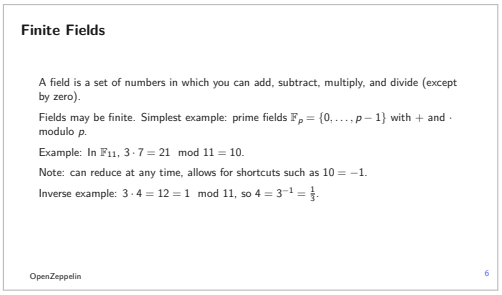
Do. Before the talk: verify or drop the Airbender row (see the cover page).

If asked. Sources for every row are on appendix slide 50. The rows have different consequences; do not say all four were exploitable soundness failures. The slide's own footnote: Airbender was one critical circuit bug and two verifier-composition flaws.

Slide 5, section title. *Circuits and Constraints.* Say: ten minutes of shared vocabulary, so every later example is precise. This is where the notation comes from.

6 Finite Fields

start **3:30** clicks **4** pdf page 10



Every value in a circuit is an element of a finite field. Our running toy is $\mathbb{F}_{11}$.

$3 \cdot 7 = 21 \equiv 10$. You may reduce whenever you like, so $10 = -1$. Division exists: $3 \cdot 4 = 12 \equiv 1$, so $\frac{1}{3} = 4$.

Two facts will matter later, so say them now:

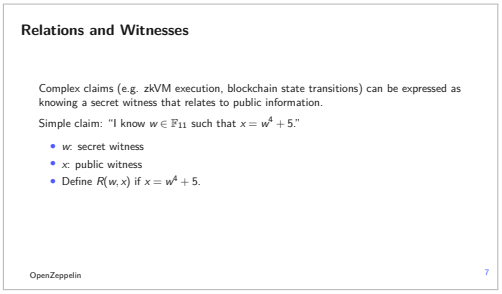
- a field has no zero divisors: $ab = 0$ forces $a = 0$ or $b = 0$;
- an equation in $\mathbb{F}_p$ is not an equation over the integers, because the field wraps around.

Do. Four clicks. Keep moving; do not teach the definition of a field.

If asked. "Why 11?" Small enough to compute by hand. Every argument today either works over any field, or we say explicitly that it does not.

7 Relations and Witnesses

start **4:30** clicks **3** pdf page 14



A zero-knowledge statement says: I know something secret that relates to something public.

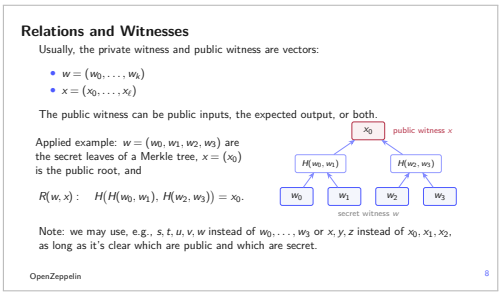
Simple claim: I know w with $x = w^4 + 5$. Here w is the secret witness, x the public witness, and the relation $R(w, x)$ is the property we actually care about.

Everything else today is one question: does the circuit really enforce R?

Do. The last click reveals nothing (a stray pause in the source). Click through it.

8 Relations and Witnesses: Merkle Example

start **5:30** clicks **3** pdf page 18



In practice both witnesses are vectors, and the public part can be inputs, outputs, or both.

Real example: four secret leaves and one public root. The relation says the leaves hash up to the root, $H(H(w_0, w_1), H(w_2, w_3)) = x_0$. Red is public, blue is secret; the deck keeps that colour convention.

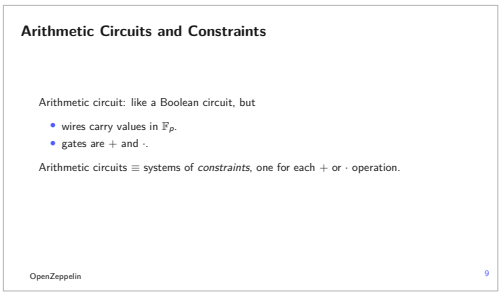
Keep this picture in mind. If a circuit computes a root but never constrains it to equal the public x_0, the proof says nothing about these leaves. That is Example 0.

Do. The tree appears on the third click. Point at the root, then the leaves.

If asked. "A Merkle proof is one leaf plus a path." Yes; the slide shows the whole tree for simplicity. The authentication-path version has the same public and secret split.

9 Arithmetic Circuits and Constraints

start **7:00** clicks **1** pdf page 20



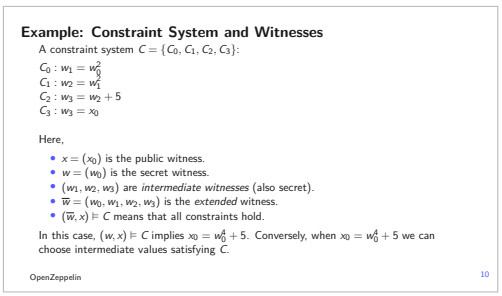
An arithmetic circuit is a Boolean circuit whose wires carry field elements and whose gates are $+$ and $\cdot$.

Each gate becomes a polynomial equation, so from here on "circuit" and "constraint system" mean the same thing: a system of polynomial equations, and only those equations are enforced.

If asked. "Is it really one constraint per gate?" Not literally. R1CS, PLONKish and AIR package gates differently. The point that survives is that the circuit is a set of polynomial equations.

10 Example: Constraint System and Witnesses

start **7:45** clicks **5** pdf page 26



Four constraints compute $w_0^4 + 5$: square twice, add five. C_3 ties the result to the public x_0.

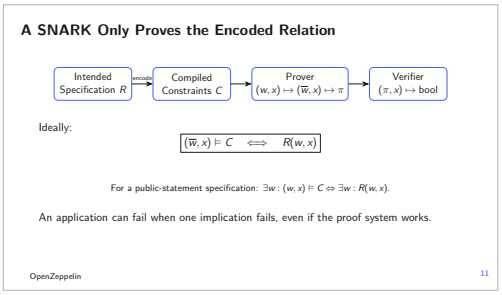
Vocabulary, one click each: x is public; $w = (w_0)$ is secret; w_1, w_2, w_3 are intermediate witnesses, also secret; $\overline{w}$ is the extended witness, everything the prover fills in; $(\overline{w}, x) \vDash C$ means every constraint holds.

Last line: for this system, satisfying C is exactly the relation. Remember C_3. In five minutes we delete it.

Do. Five clicks. Where the last line writes $(w, x) \vDash C$, say "the extended witness satisfies C".

11 A SNARK Only Proves the Encoded Relation

start **9:30** clicks **2** pdf page 29



The pipeline: an intended specification R is encoded as constraints C; the prover produces a proof; the verifier checks it.

The proof system guarantees one thing: the prover knows an extended witness satisfying C. It knows nothing about R.

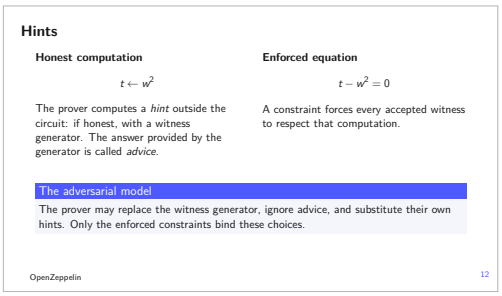
Ideally the two agree in both directions. Left to right is soundness of the circuit: everything accepted is true. Right to left is completeness: every true statement can be proven. Underconstrained means left to right fails, and that is today's topic. The footnote is the weaker, public-statement version with existential quantifiers.

Do. Gesture at the "encode" arrow: every bug today lives on that arrow.

If asked. "What about overconstrained?" That breaks completeness: honest provers get stuck. Annoying, not exploitable, which is why we focus on soundness.

12 Hints

start **11:00** clicks **0** pdf page 30



Some values are expensive or impossible to compute with $+$ and $\cdot$, such as inverses or bit decompositions. The prover computes them outside the circuit. That is a hint; the honest witness generator's answer is called advice.

Left: the honest computation $t \leftarrow w^2$. Right: what is enforced, $t - w^2 = 0$.

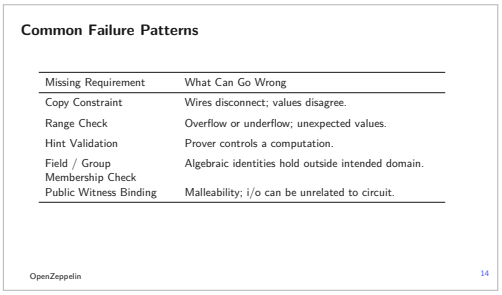
The blue box is the most important sentence today: the prover may replace the witness generator. Anything not constrained is chosen by the attacker.

A hint is not unsafe by itself. It needs enough constraints to establish the property the rest of the circuit relies on, not necessarily a unique value. Slide 21 makes that precise.

Slide 13, section title. *Kinds of Underconstraint Bugs.* Say: four worked examples, each one row of the next table.

14 Common Failure Patterns

start **12:15** clicks **0** pdf page 32



Five missing requirements:

- copy constraints: wires that should agree do not;
- range checks: field elements have no order, so "small" must be enforced;
- hint validation: the prover controls a computation;
- field or group membership: an identity holds, but in a bigger domain than intended;
- public binding: inputs or outputs float free of the circuit.

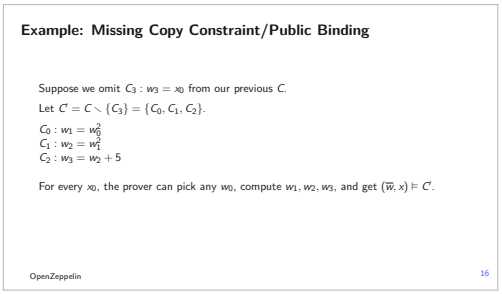
Roadmap: Example 0 is public binding, Example 1 hint validation, Exercise 2 and Example 3 range checks, and fake GLV combines a range problem with group membership. Mention disabled selectors and branch coverage. The categories overlap.

Do. One minute. It is a map, not a lecture.

Slide 15, section title. *Example 0: Missing Public Output Binding.*

16 Example: Missing Copy Constraint/Public Binding

start **13:15** clicks **2** pdf page 36



Take the constraint system from slide 10 and delete C_3.

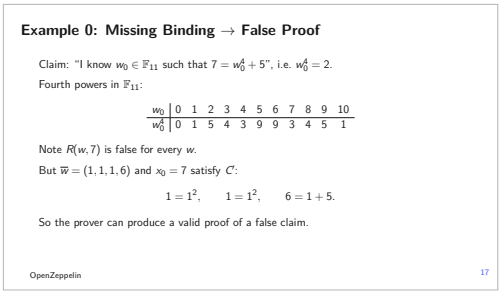
Ask the room: after the deletion, which constraint mentions x_0? None.

The private computation is still internally consistent; it computes $w_0^4 + 5$ into w_3. Nothing connects it to the public value, so for every x_0 any w_0 works.

Do. Ask the question and wait two seconds before answering.

17 Example 0: Missing Binding → False Proof

start **14:15** clicks **3** pdf page 40



A concrete false claim: I know w_0 with $7 = w_0^4 + 5$, so $w_0^4 = 2$.

The table lists every fourth power in $\mathbb{F}_{11}$: only $0, 1, 3, 4, 5, 9$. Two is not there, so no honest witness exists and the statement is false.

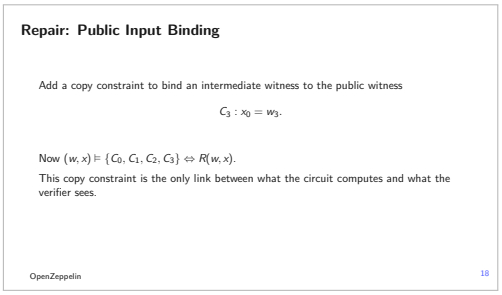
Yet $\overline{w} = (1, 1, 1, 6)$ with $x_0 = 7$ satisfies all three remaining constraints. That is a valid proof of a false statement, with the proof system working perfectly.

Do. This is the "missing public binding" line of `make demo`.

If asked. "Why so few fourth powers?" $\mathbb{F}_{11}^{\times}$ is cyclic of order 10 and $\gcd(4, 10) = 2$, so $w \mapsto w^4$ reaches only five nonzero values.

18 Repair: Public Input Binding

start **16:00** clicks **2** pdf page 43



Put the copy constraint back. Now satisfying the constraints is equivalent to the relation.

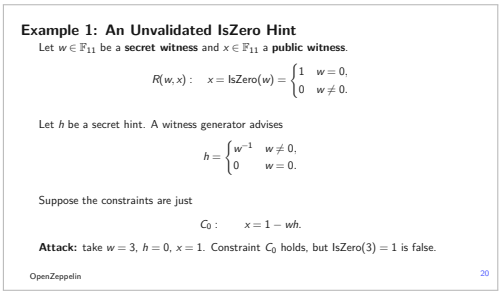
That single equation is the only link between what the circuit computes and what the verifier sees. In the Merkle picture it is the constraint that the computed root equals the public root.

Do. The title says "Input". Say "public output binding", matching the section.

Slide 19, section title. *Example 1: Unvalidated Hints.*

20 Example 1: An Unvalidated IsZero Hint

start **17:00** clicks **3** pdf page 48



IsZero is everywhere: comparisons, conditionals, "is this balance empty".

Secret w, public output x, relation $x = \text{IsZero}(w)$. IsZero cannot be computed with $+$ and $\cdot$ alone, so a hint helps: $h = w^{-1}$ when $w \neq 0$, otherwise 0. The only constraint is $x = 1 - wh$.

Honest prover: works. Attacker: take $w = 3$ and set $h = 0$. Then $x = 1$, a proof that $\text{IsZero}(3) = 1$. The honest generator never outputs $h = 0$ for nonzero w, but the attacker does not run the honest generator.

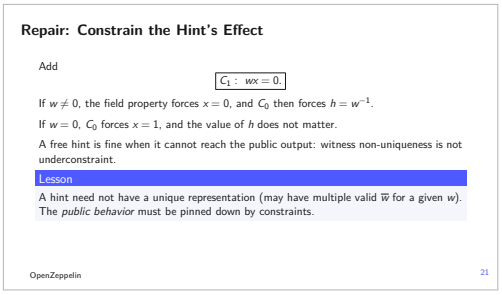
If asked. "But w is secret, and $x = 1$ is honest for $w = 0$." True for the gadget in isolation. In a real circuit w is a wire constrained elsewhere, such as a committed balance, and the gadget lets the prover output the wrong bit for that particular

w. Soundness is per witness (slide 11).

General attack: for any nonzero w and any target x, set $h = (1 - x)w^{-1}$.

21 Repair: Constrain the Hint's Effect

start **18:45** clicks **1** pdf page 50



Add $wx = 0$.

If $w \neq 0$, no zero divisors force $x = 0$, and then $x = 1 - wh$ forces $h = w^{-1}$.

If $w = 0$, the first constraint forces $x = 1$ and h can be anything. That is fine: a free hint is harmless when it cannot reach the public output. Witness non-uniqueness is not underconstraint.

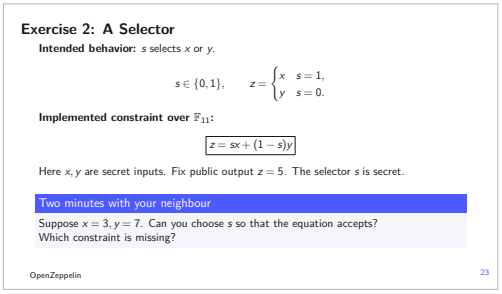
The lesson box: pin down public behaviour, not internal encodings.

If asked. "Isn't a free h a bug?" Over $\mathbb{F}_{11}$ the repaired constraints accept exactly 21 triples (w, h, x), and all 21 have the correct x; h is free only when $w = 0$. Determinism checkers such as Picus ask whether outputs are determined by inputs, and a free h does not violate that.

Slide 22, section title. *Exercise 2: A Selector.* Ask people to find a neighbour.

23 Exercise 2: A Selector

start **20:00** clicks **0** pdf page 52



Intended behaviour: s selects x or y. Implemented: one equation, $z = sx + (1 - s)y$.

Here x, y and s are secret and $z = 5$ is public. With $x = 3$ and $y = 7$: can you find an s that makes the equation accept? Which constraint is missing?

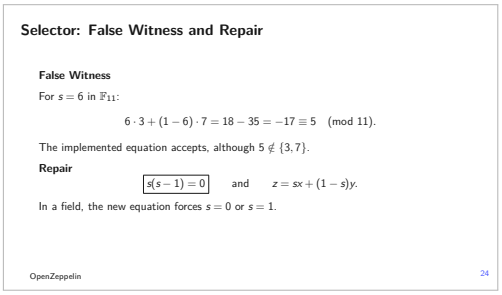
Two minutes with your neighbour.

Do. Start a visible two-minute timer. Do not advance until someone has answered. If the room is stuck, hint: solve the equation for s.

If asked. Answer key: $s = \dfrac{z - y}{x - y} = \dfrac{-2}{-4} = \dfrac{1}{2} = 6$ in $\mathbb{F}_{11}$.

24 Selector: False Witness and Repair

start **23:00** clicks **1** pdf page 54



$s = 6$: $6 \cdot 3 + (1 - 6) \cdot 7 = 18 - 35 = -17 \equiv 5$, and 5 is neither input.

A nice way to see it: 6 is $\frac{1}{2}$ in $\mathbb{F}_{11}$, and 5 is the midpoint of 3 and 7. The equation is linear interpolation; it accepts every point on the line through x and y.

Repair: $s(s - 1) = 0$. No zero divisors, so $s \in \{0, 1\}$ and the output is x or y. The argument uses nothing about 11: it holds in every field, and the repository has it as a machine-checked Lean theorem, `selector_sound`.

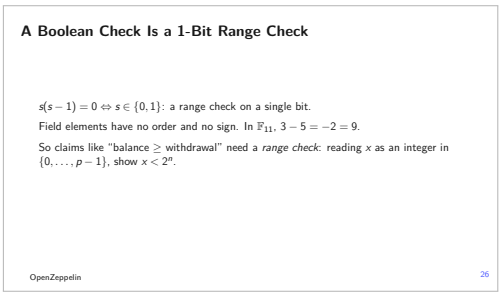
Do. Optional: show the "non-Boolean selector" line of `make demo`, or open `lean/Workshop/Selector.lean`.

If asked. "Why every field?" Only the zero-product rule is used. In $\mathbb{Z}/6$, $s = 3$ satisfies $s(s - 1) = 0$ and breaks the selector. For "but x, y are secret", give the IsZero answer (slide 20).

Slide 25, section title. *Example 3: Range Checks and Foreign-Field Arithmetic.* Say: this is the bridge to the gnark case study.

26 A Boolean Check Is a 1-Bit Range Check

start **25:00** clicks **2** pdf page 58

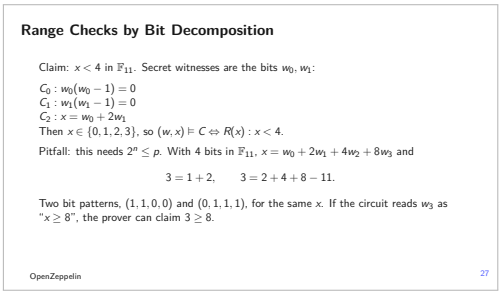


$s(s-1)=0$ is a range check on a single bit.

Fields have no order and no sign: in $\mathbb{F}_{11}$, $3-5=9$. So any claim like "balance $\geq$ withdrawal" needs an explicit range check: read the value as an integer in $\{0,\ldots,p-1\}$ and prove it is below 2^n.

27 Range Checks by Bit Decomposition

start **26:00** clicks **2** pdf page 61



To show $x<4$: two Boolean bits and $x=w_0+2w_1$, so $x \in \{0,1,2,3\}$.

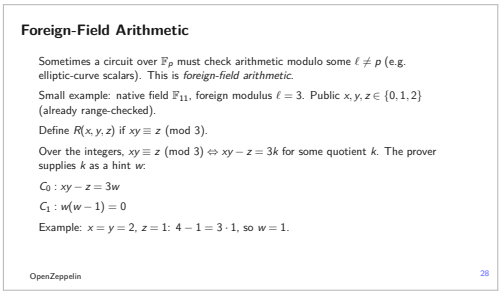
Pitfall: this needs $2^n \leq p$. With four bits in $\mathbb{F}_{11}$, $3=1+2$ and also $3=2+4+8-11$: two bit patterns for the same x. A circuit that reads the top bit as "$x \geq 8$" lets the prover claim $3 \geq 8$.

This is circomlib's Num2Bits pattern. A CAV 2024 crawl found bit-sums in 98% of public Circom projects. It is also the pattern that made SMT solvers time out; we come back to that on slide 43.

Do. If behind schedule, give only the pitfall sentence.

28 Foreign-Field Arithmetic

start **27:30** clicks **3** pdf page 65



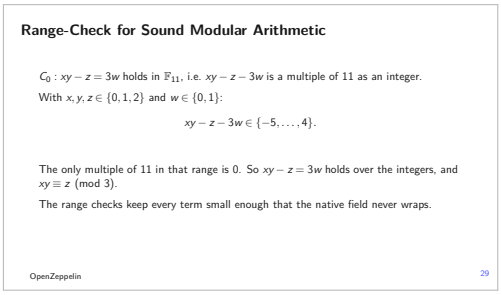
Sometimes the circuit lives in $\mathbb{F}_p$ but must check arithmetic modulo a different number ℓ. Elliptic-curve scalars are the standard case.

Toy: native $\mathbb{F}_{11}$, foreign modulus 3, public $x,y,z \in \{0,1,2\}$ already range-checked. We want $xy \equiv z \pmod 3$, that is $xy-z=3k$ over the integers.

The prover supplies the quotient k as a hint w; we constrain $xy-z=3w$ and w Boolean. Example: $2\cdot 2=4\equiv 1$, quotient 1.

29 Range-Check for Sound Modular Arithmetic

start **28:45** clicks **3** pdf page 69



Why is this sound? The equation holds in $\mathbb{F}_{11}$, so $xy-z-3w$ is a multiple of 11 as an integer.

With the ranges, that integer lies in $\{-5,\ldots,4\}$, and the only multiple of 11 there is 0. So the equation holds over the integers, and the congruence mod 3 follows.

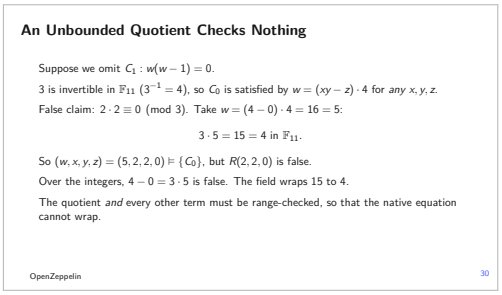
The range checks are what let you lift a field equation to an integer equation. Remember this argument: it is exactly what the gnark repair needs, and it is the kind of reasoning SMT solvers find hardest.

Do. If behind schedule, give only the last paragraph.

If asked. "Does a Boolean quotient suffice for real curve arithmetic?" No. Real gadgets use limbs and carries with bounds on every term; a Boolean quotient works here only because the numbers are tiny.

30 An Unbounded Quotient Checks Nothing

start **30:00** clicks **4** pdf page 74



Now drop the Boolean constraint on w. Since 3 is invertible in $\mathbb{F}_{11}$, $w = (xy - z) \cdot 3^{-1} = (xy - z) \cdot 4$ satisfies the equation for any x, y, z.

False claim: $2 \cdot 2 \equiv 0 \pmod 3$. Take $w = 5$: $3 \cdot 5 = 15 = 4$ in $\mathbb{F}_{11}$, so it passes. Over the integers $4 = 15$ is false; the field wrapped around.

Moral: the quotient and every other term must be range-checked.

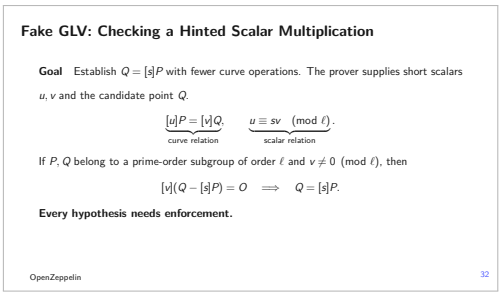
Transition: now the same bug inside a widely used library.

Do. This is the "unbounded foreign-field quotient" line of `make demo`.

Slide 31, section title. *Deep Dive: gnark Fake GLV.* Say: everything we need is already on the board: two fields, a quotient, and a cancellation.

32 Fake GLV: Checking a Hinted Scalar Multiplication

start **31:15** clicks **0** pdf page 76



Goal: prove $Q = [s]P$ cheaply. Instead of computing $[s]P$ in the circuit, the prover supplies Q and two short scalars u, v. The circuit checks a curve relation $[u]P = [v]Q$ and a scalar relation $u \equiv sv \pmod{\ell}$. Short scalars roughly halve the cost of the curve check.

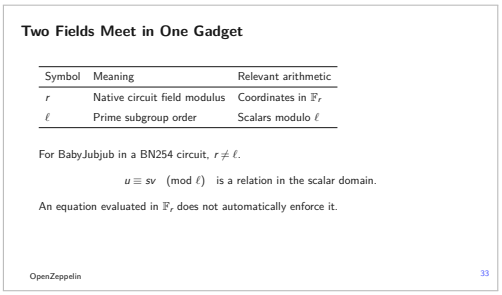
Correctness: substitute to get $[v](Q - [s]P) = O$, then cancel v. Cancelling needs two hypotheses: P, Q in the prime-order subgroup, and $v \not\equiv 0 \pmod{\ell}$.

"Every hypothesis needs enforcement" is the checklist for the next five slides.

If asked. "Why are the scalars short?" Appendix slide 53: lattice reduction outside the circuit gives $|u|, |v| \approx \sqrt{\ell}$.

33 Two Fields Meet in One Gadget

start **32:30** clicks **0** pdf page 77



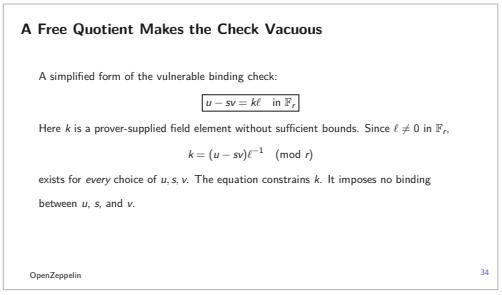
Two moduli. r is the native field of the circuit, where curve coordinates live. ℓ is the subgroup order, where scalars live.

For BabyJubjub inside a BN254 circuit, coordinates are native but $\ell \neq r$. So $u \equiv sv \pmod{\ell}$ is foreign-field arithmetic: slide 28 with ℓ in place of 3. An equation in $\mathbb{F}_r$ does not enforce it.

If asked. Do not conflate coordinate arithmetic, which is native here, with scalar arithmetic, which is foreign.

34 A Free Quotient Makes the Check Vacuous

start **33:30** clicks **0** pdf page 78



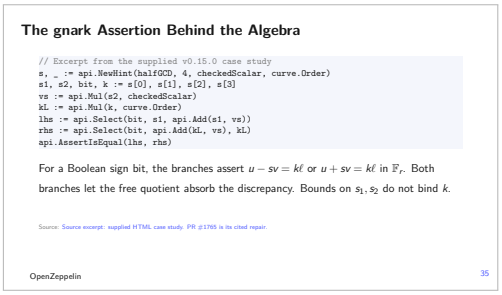
The vulnerable pattern checks $u - sv = k\ell$ in $\mathbb{F}_r$, with k a prover-supplied hint and no bound on it.

Since $\ell \neq 0$ in $\mathbb{F}_r$, $k = (u - sv)\ell^{-1}$ exists for every u, s, v. The equation constrains k and nothing else.

This is slide 30 at 254 bits.

35 The gnark Assertion Behind the Algebra

start **34:30** clicks **0** pdf page 79


The gnark Assertion Behind the Algebra

```
// Excerpt from the supplied v0.15.0 case study
s, _ := api.NewHint(halfGCD, 4, checkedScalar, curve.Order)
s1, s2, bit, k := s[0], s[1], s[2], s[3]
vs := api.Mul(s2, checkedScalar)
kL := api.Mul(k, curve.Order)
lhs := api.Select(bit, s1, api.Add(s1, vs))
rhs := api.Select(bit, api.Add(kL, vs), kL)
api.AssertIsEqual(lhs, rhs)
```

For a Boolean sign bit, the branches assert $u - sv = k\ell$ or $u + sv = k\ell$ in $\mathbb{F}_\ell$. Both branches let the free quotient absorb the discrepancy. Bounds on s_1, s_2 do not bind k.

Source: Source excerpt: supplied HTML case study. PR #1765 is its cited repair.

OpenZeppelin 35

Walk the code top to bottom. The hint returns four values: s_1, s_2 (our u, v), a sign bit, and the quotient k. The two Selects implement the two sign branches, and AssertIsEqual checks $u \mp sv = k\ell$ natively.

s_1 and s_2 are range-checked to half the bit length. k is never bounded, so in both branches k absorbs any discrepancy.

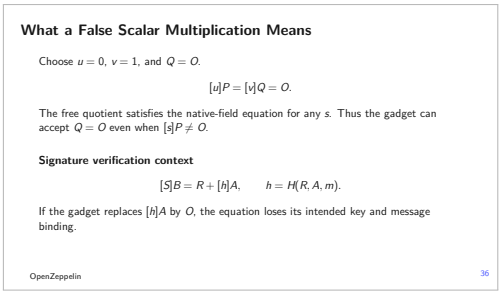
Do. **Checked against gnark's source (28 Oct 2026).** The excerpt matches `std/algebra/native/twistededwards/point.go` at **v0.15.0** (released 13 May 2026), lines 135–147, with variables renamed. In v0.15.0 the public `ScalarMul` for native twisted Edwards curves calls this function directly. The very next line in v0.15.0 already rejects $s_2 = 0$, which is why slide 36 uses $v = 1$ rather than $v = 0$.

Before the talk: replace the on-slide source line "supplied HTML case study".

If asked. "Which versions? Was it exploited?" State only what the source shows: v0.15.0 has the unbounded quotient; v0.16.0 (19 Aug 2026) replaces it (slide 37). PR #1765, a performance PR merged 15 July 2026, describes wiring scalar-decomposition verification into this path and adds regression tests for malicious decompositions and forged hints. Make no claim about exploitation.

36 What a False Scalar Multiplication Means

start **36:00** clicks **0** pdf page 80


What a False Scalar Multiplication Means

Choose $u = 0$, $v = 1$, and $Q = O$.

$$[u]P = [v]Q = O.$$

The free quotient satisfies the native-field equation for any s. Thus the gadget can accept $Q = O$ even when $[s]P \neq O$.

Signature verification context

$$[S]B = R + [h]A, \qquad h = H(R, A, m).$$

If the gadget replaces $[h]A$ by O, the equation loses its intended key and message binding.

OpenZeppelin 36

Concrete attack: $u = 0$, $v = 1$, $Q = O$. The curve relation $[0]P = [1]O$ holds trivially, and the free quotient satisfies the scalar relation for any s. So the gadget accepts the identity O as $[s]P$ for every scalar.

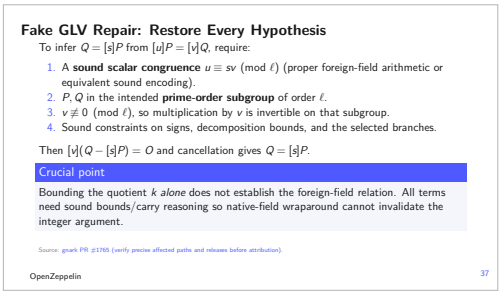
Why that matters: signature verification checks $[S]B = R + [h]A$ with $h = H(R, A, m)$. If $[h]A$ comes from this gadget and the prover sets it to O, the check becomes $[S]B = R$, which anyone satisfies by choosing S and setting $R = [S]B$. The key and the message drop out.

Do. This attack follows from reading the v0.15.0 code on slide 35. It was derived from the source, not run against gnark.

If asked. "So every user was forgeable?" No such claim. Reachability depends on how an application called the gadget and which version it pinned.

37 Fake GLV Repair: Restore Every Hypothesis

start **37:00** clicks **0** pdf page 81


Fake GLV Repair: Restore Every Hypothesis

To infer $Q = [s]P$ from $[u]P = [v]Q$, require:

1. A **sound scalar congruence** $u \equiv sv \pmod{\ell}$ (proper foreign-field arithmetic or equivalent sound encoding).
2. P, Q in the intended **prime-order subgroup** of order ℓ.
3. $v \not\equiv 0 \pmod{\ell}$, so multiplication by v is invertible on that subgroup.
4. Sound constraints on signs, decomposition bounds, and the selected branches.

Then $[v](Q - [s]P) = O$ and cancellation gives $Q = [s]P$.

Crucial point

Bounding the quotient k *alone* does not establish the foreign-field relation. All terms need sound bounds/carry reasoning so native-field wraparound cannot invalidate the integer argument.

Source: gnark PR #1765 (verify precise affected paths and releases before attribution).

OpenZeppelin 37

The checklist from slide 32:

1. a sound scalar congruence, real foreign-field arithmetic rather than a free quotient;
2. P, Q in the prime-order subgroup;
3. $v \not\equiv 0 \pmod{\ell}$;
4. bounds on signs, decomposition and branches.

With all four, $[v](Q - [s]P) = O$ and cancellation gives $Q = [s]P$.

The crucial-point box: bounding k alone is not enough. Every term needs bounds, exactly as on slide 29.

Do. **gnark v0.16.0 implements this list** (checked in its source). The decomposition is verified with emulated arithmetic modulo the curve order, so the native quotient is gone (item 1). $s_2 \neq 0$ is asserted in that emulated field (item 3). The hinted point is forced into the prime-order subgroup, with a source comment describing a 2-torsion attack otherwise (item 2). s_1, s_2 are range-checked (item 4).

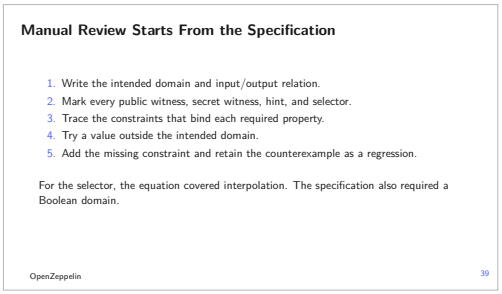
Before the talk: replace "verify precise affected paths and releases" in the source line.

If asked. "Is the cancellation step proved?" Yes, in the model: the repository's Lean theorem `scalar_cancellation` proves it over any vector space, from the scalar relation and $v \neq 0$. Item 2 is what makes the prime-order group a vector space over $\mathbb{F}_\ell$.

Slide 38, section title. *Finding Underconstraints.* Say: we have seen five bugs. How would you have found them?

39 Manual Review Starts From the Specification

start **38:15** clicks **0** pdf page 83



Five steps:

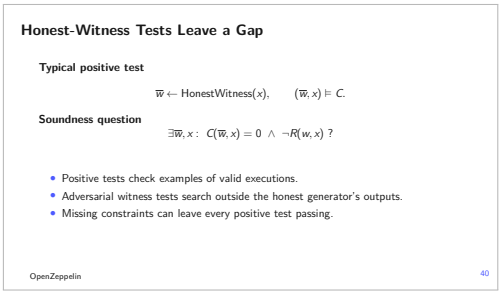
1. write the intended domain and relation before reading constraints;
2. mark every value as public, secret, hint or selector;
3. for each property the specification needs, find the constraint that enforces it;
4. try a value outside the intended domain;
5. add the missing constraint and keep the counterexample as a regression test.

For the selector: the equation covered the interpolation, but the specification also required s Boolean, and nothing enforced that.

If asked. Reviewers must also read call sites and dependency assumptions. Fake GLV was a library bug, invisible from application code.

40 Honest-Witness Tests Leave a Gap

start **39:15** clicks **0** pdf page 84



A typical test generates the witness honestly and checks that the constraints hold. That tests completeness.

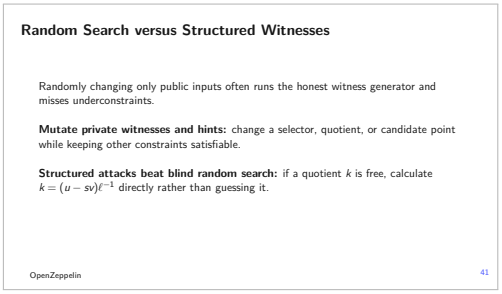
The soundness question is different: does any extended witness satisfy C but violate R?

Every example today passes every honest test, because the honest generator never produces $s = 6$ or $h = 0$.

If asked. Do not say testing is useless: negative tests with injected witnesses do catch these bugs.

41 Random Search versus Structured Witnesses

start **40:00** clicks **0** pdf page 85



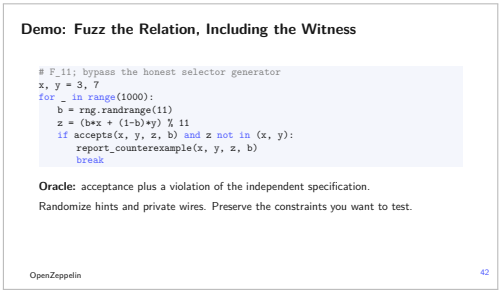
Fuzzing public inputs just reruns the honest generator. Fuzz the private witness and the hints instead.

Sometimes random works: for the selector, 9 of the 11 values of s in $\mathbb{F}_{11}$ are counterexamples, and over a large field almost all of them are.

Sometimes it cannot: a random k in a 254-bit field essentially never equals $(u - sv)\ell^{-1}$, but you can compute that value directly. Structure beats luck.

42 Demo: Fuzz the Relation, Including the Witness

start **40:45** clicks **0** pdf page 86



The oracle is the independent specification: the constraints accept, and $z \notin \{x, y\}$.

Mutate private wires and hints; keep the constraints you are testing.

Then switch to the terminal and show it.

Do. From the repository root run `make demo`. Point at the non-Boolean selector entry: witness $(3, 7, 5; 6)$, vulnerable ACCEPT, repaired REJECT. Optionally `make test`: seven tests, exhaustive over $\mathbb{F}_{11}$.

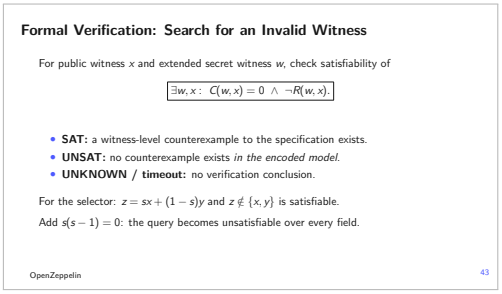
The loop on the slide is illustrative and is not in the repository. As written, `accepts` is automatically true, because z is computed from the constraint.

Fallback without a terminal: slide 24 already shows the result.

If asked. The demo prints x, y, z as public, while the slides make x, y secret. The arithmetic is identical.

43 Formal Verification: Search for an Invalid Witness

start **42:45** clicks **1** pdf page 88



Soundness is a "for all": every accepted witness satisfies R. A solver proves it by refuting the negation: is there a witness with $C = 0$ and not R?

- SAT returns a concrete exploit;
- UNSAT means no counterexample exists in the encoded model;
- UNKNOWN or timeout means nothing.

We ran the selector in cvc5's finite-field theory. Without booleanity: SAT, and pinned to $(3, 7, 5)$ it finds $s = 6$ in milliseconds. With $s(s-1) = 0$: UNSAT, also over the 254-bit BN254 field.

This is not fuzzing. UNSAT comes from algebra: a certificate that 1 lies in the ideal generated by the constraints.

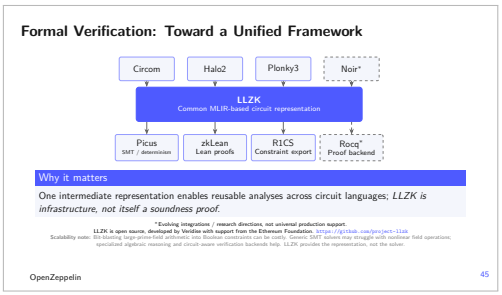
Do. The companion note `notes/smt-ring-theory.pdf` has the certificate and the split Gröbner basis story. Commit it before mentioning it.

If asked. "Why not always use SMT?" Gröbner-basis reasoning is worst-case doubly exponential. Bit decompositions (slide 27) were the classic blow-up; split Gröbner bases (CAV 2024) raised solved benchmarks from 475 to 969 of 1140, but integer division and remainder, the argument of slide 29, remain hard. That is where Lean proofs take over.

Slide 44, section title. *Verification Infrastructure and AI.*

45 Formal Verification: Toward a Unified Framework

start **44:15** clicks **0** pdf page 90



Problem: every circuit language would need its own analysers.

LLZK is a common MLIR-based representation. Circom, Halo2 and Plonky3 compile into it, and analyses run on it: Picus for SMT-based determinism, zkLean for Lean proofs, R1CS export. Dashed boxes are research directions, not production support.

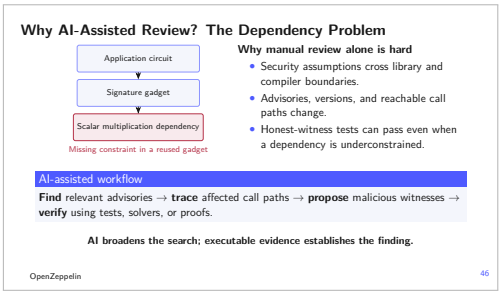
The box says it plainly: LLZK is infrastructure, not itself a soundness proof.

Credit: open source, developed by Veridise with support from the Ethereum Foundation.

If asked. "What about Zirgen?" RISC Zero's circuit compiler; Veridise has a zirgen-to-LLZK frontend. Zirgen writes constraints, solvers decide them: compiler versus decision procedure.

46 Why AI-Assisted Review? The Dependency Problem

start **45:30** clicks **0** pdf page 91



The fake-GLV story as a picture: the application circuit is fine, the signature gadget is fine, and the bug sits two layers down in a reused scalar-multiplication dependency.

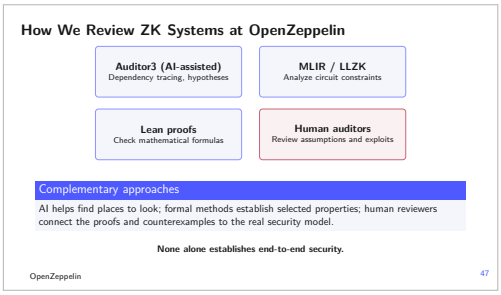
Manual review alone struggles: assumptions cross library and compiler boundaries, and advisories, versions and call paths keep changing.

AI helps find where to look: advisories, call paths, candidate malicious witnesses. The last step is non-negotiable: verify with tests, solvers or proofs. Read the bottom line as written.

If asked. Present AI as triage, not as proof of an exploit and not as a substitute for human analysis.

47 How We Review ZK Systems at OpenZeppelin

start **46:30** clicks **0** pdf page 92



Four complementary approaches: Auditor3, AI-assisted, for dependency tracing and hypotheses; MLIR and LLZK to analyse circuit constraints; Lean to check the mathematics; human auditors to connect proofs and counterexamples to the real threat model.

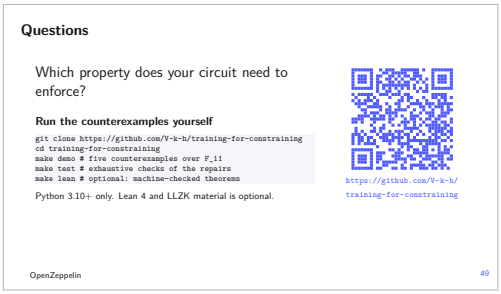
None alone establishes end-to-end security.

If asked. Careful wording: not every engagement uses every technique.

Slide 48, section title. *Wrap-Up.*

49 Questions

start **47:30** clicks **0** pdf page 94



Ask the question on the slide: which property does your circuit need to enforce?

Then the takeaway: a prover can replace every private witness and every hint. Only enforced constraints bind the public claim. Check the assumptions of your dependencies and compilers, not only your own code.

Everything today runs from this repository with Python alone.

Do. Leave this slide up through questions so people can scan the QR code. The repository must be public by now.

If asked. **Must a hint have a unique value?** No (slide 21). **Does an on-curve check imply subgroup membership?** Not in general; gnark v0.16.0 added an explicit subgroup check for that reason. **Why is a SAT witness better than a warning?** It is an executable counterexample that becomes a regression test.

50 Appendix: Source Reports

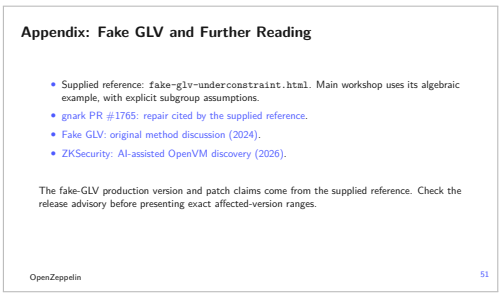
start **backup** clicks **0** pdf page 95



Use only if asked about the incidents on slide 4. Every link on this slide resolved when checked on 28 October 2026.

51 Appendix: Fake GLV and Further Reading

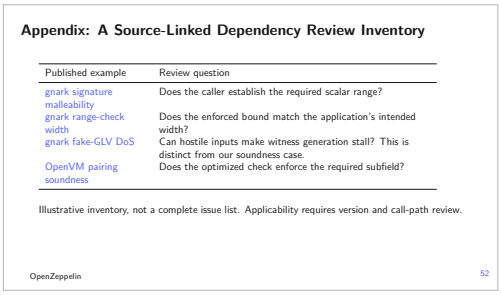
start **backup** clicks **0** pdf page 96



Further reading on fake GLV, including the original ethresear.ch method write-up.

Do. Before the talk: replace "Supplied reference: fake-glv-underconstraint.html" and "Check the release advisory" with the gnark facts on the pages for slides 35 and 37.

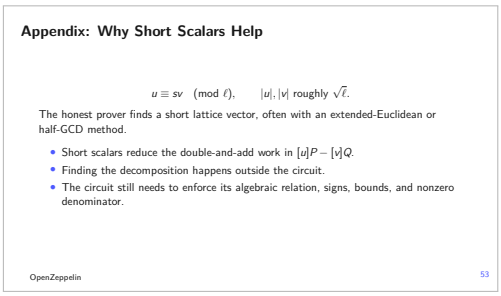
52 Appendix: A Source-Linked Dependency Review Inventory



Example review questions per published advisory. Use it if someone asks how to start a dependency review.

If asked. The fake-GLV DoS advisory listed here is a different issue from the soundness case in the main deck.

53 Appendix: Why Short Scalars Help



For "why short scalars". Lattice reduction outside the circuit finds $u \equiv sv \pmod{\ell}$ with $|u|, |v| \approx \sqrt{\ell}$, which halves the double-and-add loop. Because the decomposition happens outside the circuit, the circuit must enforce the relation, signs, bounds and a nonzero v.

If asked. gnark's v0.15.0 hint used a half-GCD lattice decomposition; v0.16.0 uses LLL-based rational reconstruction.